

Week 2 Assignment — Java Code Answers

Complete solutions for all 5 problems

1. ATM PIN Length Validator

Checks that the PIN string contains exactly four characters, as required by the assignment.

```
public static void checkPinLength(String pin) {
    if (pin.length() != 4) {
        System.out.println("Invalid PIN – must be exactly 4 digits.");
    } else {
        System.out.println("PIN length OK.");
    }
}
```

2. Word Reversal Encoder

Reverses each word separately and keeps the original word order.

```
public static String reverseEachWord(String sentence) {
    String[] words = sentence.split(" ");
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < words.length; i++) {
        StringBuilder reversed = new StringBuilder();
        for (int j = words[i].length() - 1; j >= 0; j--) {
            reversed.append(words[i].charAt(j));
        }

        if (i > 0) {
            result.append(" ");
        }
        result.append(reversed);
    }

    return result.toString();
}
```

3. Product Inventory CSV Parser

Splits the CSV line into fields, checks for exactly three fields, then prints the formatted record.

```
public static void parseInventoryRecord(String csvLine) {
    String[] fields = csvLine.split(",", -1);

    if (fields.length != 3) {
        System.out.println("Invalid Record");
    } else {
        System.out.println("Product: " + fields[0].trim()
            + " | SKU: " + fields[1].trim()
            + " | Qty: " + fields[2].trim());
    }
}
```

4. Library ISBN Normalizer & Validator

Normalizes the publisher code and validates the length, publisher letters, and numeric body. This follows the assignment's 3-letter + 4-digit year + 6-digit catalog format.

```
public static String normalizeCode(String raw) {
    String code = raw.trim();

    if (code.length() < 3) {
        return code.toUpperCase();
    }

    return code.substring(0, 3).toUpperCase() + code.substring(3);
}

public static String validateAndFormat(String code) {
    if (code.length() != 13) {
        return "Invalid: code must be exactly 13 characters";
    }
}
```

```

    for (int i = 0; i < 3; i++) {
        if (!Character.isLetter(code.charAt(i))) {
            return "Invalid: publisher code must be 3 letters";
        }
    }

    for (int i = 3; i < 13; i++) {
        if (!Character.isDigit(code.charAt(i))) {
            return "Invalid: body must contain only digits";
        }
    }

    String year = code.substring(3, 7);
    String catalog = code.substring(7, 13);

    StringBuilder result = new StringBuilder();
    result.append("[").append(code.substring(0, 3)).append("] ");
    result.append("YEAR: ").append(year).append(" | CATALOG: ");
    result.append(catalog);

    return result.toString();
}

// Example usage:
// String code = normalizeCode(" pen2026004251 ");
// System.out.println(validateAndFormat(code));

```

5. Stop-Word-Filtered Word Frequency Report

Removes the specified stop words, counts the remaining words, and prints counts from highest to lowest.

```

import java.util.*;

public static void printFilteredWordFrequency(String feedback) {
    String cleaned = feedback.toLowerCase()
        .replace(".", "")
        .replace(",", "");

    String[] words = cleaned.trim().split("\\s+");
    Set<String> stopWords = new HashSet<>(Arrays.asList(
        "the", "was", "and", "a", "is", "of", "in"));

    Map<String, Integer> frequency = new HashMap<>();

    for (String word : words) {
        if (!word.isEmpty() && !stopWords.contains(word)) {
            frequency.put(word, frequency.getOrDefault(word, 0) + 1);
        }
    }

    List<Map.Entry<String, Integer>> entries =
        new ArrayList<>(frequency.entrySet());

    entries.sort((e1, e2) ->
        e2.getValue().compareTo(e1.getValue()));

    for (Map.Entry<String, Integer> entry : entries) {
        System.out.println(entry.getKey() + ": " + entry.getValue());
    }
}

```

Note: For problem 5, add `import java.util.*;` at the top of your Java file. The other snippets can be placed inside a class. For problem 4, the sample code `pen2026004251` is 13 characters and is formatted as `[PEN] YEAR: 2026 | CATALOG: 004251`.